CSC 212

Ride-Sharing System - Phase 1

Team Members

Student name	University ID
نواف تركي القصير	446107045
بسام فيصل الزهراني	446106881
وليد داني حلبي	446109612

Division of Work

نواف تركي القصير | 446107045 | Person, Driver, DriverList, DateTime

بسام فيصل الزهراني | 446106881 | Main, RideSharingSystem

وليد داني حلبي | 446109612 | LinkedList, Node, RiderList, Rider, RideList, Ride, PrivateRide, SharedRide

Analysis — نواف تركي القصير

Person

Person (constructor)

CODE	S/E	FREQUENCY	TOTAL
// Student Name: نواف تركي القصير	0	0	0
// Student ID: 446107045	0	0	0
public abstract class Person implements IPerson {	0	0	0
private final int id;	0	0	0
private String name;	0	0	0
private String phoneNumber;	0	0	0
private LinkedList<IRide> rideHistory;	0	0	0
//o(1) time complixty	0	0	0
public Person(int id, String name, String phoneNumber) {	0	0	0
this.id = id;	1	1	1
this.name = name;	1	1	1
setPhoneNumber(phoneNumber);	1	1	1
this.rideHistory = new LinkedList<>();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4**		
BIG O	**O(1)**		

getId()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public int getId() {	0	0	0
return id;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getName()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public String getName() {	0	0	0
return name;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setName()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public void setName(String name) {	0	0	0
this.name = name;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getPhoneNumber()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public String getPhoneNumber() {	0	0	0
return phoneNumber;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setPhoneNumber()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public void setPhoneNumber(String phoneNumber) {	0	0	0
if(phoneNumber==null || phoneNumber.length()!=10) {	1	1	1
throw new IllegalArgumentException("Phone number must be exactly 10 digits!");	0	0	0
}// restricting the user to enter a phone number exactly contains 10 digits	0	0	0
char c;	1	1	1
for(int i=0;i<phoneNumber.length();i++) {	1	11	11
c=phoneNumber.charAt(i);	1	10	10
if(c<'0' ||c>'9')	1	10	10
throw new IllegalArgumentException("Phone number must contain digits only");	0	0	0
} // not allowing a letter to be part of the phone number	0	0	0
this .phoneNumber=phoneNumber;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 34**		
BIG O	**O(1)**		

getRideHistory()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public LinkedList<IRide> getRideHistory() {	0	0	0
return rideHistory;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

toString()

CODE	S/E	FREQUENCY	TOTAL
public String toString() {	0	0	0
return "ID: " + id +	1	1	1
", Name: " + name +	0	0	0
", Phone: " + phoneNumber;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2**		
BIG O	**O(1)**		

Driver

Driver (constructor)

CODE	S/E	FREQUENCY	TOTAL
// Student Name: نواف تركي القصير	0	0	0
// Student ID: 446107045	0	0	0
public class Driver extends Person implements IDriver {	0	0	0
private String vehiclePlate;	0	0	0
private VehicleType vehicleType;	0	0	0
//o(1) time complixty	0	0	0
public Driver(int id, String name, String phoneNumber,	0	0	0
String vehiclePlate, VehicleType vehicleType) {	0	0	0
super(id, name, phoneNumber);	1	1	1

CODE	S/E	FREQUENCY	TOTAL
setVehiclePlate(vehiclePlate);	1	1	1
setVehicleType(vehicleType);	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3**		
BIG O	**O(1)**		

getVehiclePlate()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public String getVehiclePlate()	0	0	0
{	0	0	0
return vehiclePlate;	1	1	1
} //o(1) time complixty	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setVehiclePlate()

CODE	S/E	FREQUENCY	TOTAL
public void setVehiclePlate(String vehiclePlate){	0	0	0
// Check the length	0	0	0
if (vehiclePlate == null || vehiclePlate.length() != 7)	1	1	1
{	0	0	0
throw new IllegalArgumentException("Vehicle plate must contain exactly 7 characters");	0	0	0
}	0	0	0
for(int i=0;i<3;i++) //for cheaking the first 3 letters	1	4	4
{	0	0	0
if(!(vehiclePlate.charAt(i)>=65 && vehiclePlate.charAt(i)<=90)) //asci code upper letters from 65 to 90	1	3	3
throw new IllegalArgumentException("First 3 characters must be uppercase letters");	0	0	0
}	0	0	0
for(int i=3 ;i<7;i++)	1	5	5
{	0	0	0
char cDigit=vehiclePlate.charAt(i);	1	4	4
if(!(cDigit<='9' && cDigit>='0'))	1	4	4

CODE	S/E	FREQUENCY	TOTAL
throw new IllegalArgumentException("Last 4 characters must be digits"); }	0	0	0
this.vehiclePlate = vehiclePlate;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 22**		
BIG O	**O(1)**		

getVehicleType()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public VehicleType getVehicleType() {	0	0	0
return vehicleType;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setVehicleType()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public void setVehicleType(VehicleType vehicleType) {	0	0	0
if (vehicleType == null) {	1	1	1
throw new IllegalArgumentException(	0	0	0
"Vehicle type cannot be null");	1	1	1
}	0	0	0
this.vehicleType = vehicleType;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3**		
BIG O	**O(1)**		

equals()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public boolean equals(Object obj) {	0	0	0
if (this == obj) {	1	1	1
return true;	0	0	0
}	0	0	0
if (!(obj instanceof IDriver)) {	1	1	1
return false;	0	0	0
}	0	0	0
IDriver other = (IDriver) obj;	1	1	1
return this.getId() == other.getId();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4**		
BIG O	**O(1)**		

compareTo()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public int compareTo(IDriver other) {	0	0	0
if (this.getId() > other.getId()) {	1	1	1
return 1;	0	0	0
}	0	0	0
else if (this.getId() < other.getId()) {	1	1	1
return -1;	0	0	0
}	0	0	0
else {	0	0	0
return 0;	1	1	1
}	0	0	0
} //o(1) time complixty	0	0	0
TOTAL OPERATIONS	**T(n) = 3**		
BIG O	**O(1)**		

hashCode()

CODE	S/E	FREQUENCY	TOTAL
public int hashCode() {	0	0	0
return getId();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

toString()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time complixty	0	0	0
public String toString() {	0	0	0
return super.toString()	1	1	1
+ ", Vehicle Plate: " + vehiclePlate	0	0	0
+ ", Vehicle Type: " + vehicleType;	0	0	0
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

DriverList

DriverList (constructor)

CODE	S/E	FREQUENCY	TOTAL
// Student Name : نواف تركي القصير	0	0	0
// Student Id :446107045	0	0	0
public class DriverList implements IDriverList	0	0	0
{	0	0	0
private Node<IDriver> head;	0	0	0
private int size;	0	0	0
//o(1) time complixity	0	0	0
public DriverList() {	0	0	0

CODE	S/E	FREQUENCY	TOTAL
head=null;	1	1	1
size=0;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2**		
BIG O	**O(1)**		

add()

CODE	S/E	FREQUENCY	TOTAL
//worst time complixity is o(n)	0	0	0
public boolean add(IDriver driver) {	0	0	0
if (driver == null)	1	1	1
return false;	0	0	0
Node<IDriver> check = head;	1	1	1
// cheaking Whether the same vehcile plate existed	0	0	0
// avoiding duplicates	0	0	0
while (check != null) {	1	n+1	n+1
if (check.data.compareTo(driver) == 0 ||	1	n	n
check.data.getVehiclePlate().equals(driver.getVehiclePlate())) {	0	0	0
return false;	0	0	0
}	0	0	0
check = check.next;	1	n	n
}	0	0	0
// if the list of drivers are empty it will be created at first	0	0	0
if (head == null) {	1	1	1
Node<IDriver> d = new Node<>(driver);	0	0	0
head = d;	0	0	0
size++;	0	0	0
return true;	0	0	0
}	0	0	0
if (head.data.compareTo(driver) > 0) {	1	1	1
Node<IDriver> d = new Node<>(driver);	0	0	0
d.next = head;	0	0	0
head = d;	0	0	0
size++;	0	0	0
return true;	0	0	0

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
Node<IDriver> current = head;	1	1	1
Node<IDriver> prev = null;	1	1	1
while (current != null) {	1	n+1	n+1
if (current.data.compareTo(driver) < 0) {	1	n	n
prev = current;	1	n	n
current = current.next;	1	n	n
}	0	0	0
else {	0	0	0
Node<IDriver> d = new Node<>(driver);	0	0	0
d.next = current;	0	0	0
prev.next = d;	0	0	0
size++;	0	0	0
return true;	0	0	0
}	0	0	0
}	0	0	0
Node<IDriver> d = new Node<>(driver);	1	1	1
prev.next = d;	1	1	1
size++;	1	1	1
return true;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 7*n + 12**		
BIG O	**O(n)**		

findById()

CODE	S/E	FREQUENCY	TOTAL
//worst time complixity is o(n)	0	0	0
public IDriver findById(int driverId) {	0	0	0
Node<IDriver>current=head;	1	1	1
while(current!=null)	1	n+1	n+1
{	0	0	0
if(current.data.getId()==driverId)	1	n	n
return current.data;	0	0	0
current=current.next;	1	n	n

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
return null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 3**		
BIG O	**O(n)**		

findByName()

CODE	S/E	FREQUENCY	TOTAL
//worst time complixity is o(n)	0	0	0
public LinkedList<IDriver> findByName(String fullName) {	0	0	0
LinkedList<IDriver>ListOfNames=new LinkedList<>();	1	1	1
Node<IDriver>current=head;	1	1	1
while(current!=null) {	1	n+1	n+1
if(current.data.getName().equals(fullName))	1	n	n
ListOfNames.insert(current.data);	1	n	n
current=current.next;	1	n	n
}	0	0	0
return ListOfNames;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 4**		
BIG O	**O(n)**		

findByVehiclePlate()

CODE	S/E	FREQUENCY	TOTAL
//worst time complixity is o(n)	0	0	0
public IDriver findByVehiclePlate(String vehiclePlate) {	0	0	0
Node<IDriver>current =head;	1	1	1
while(current!=null) {	1	n+1	n+1
if(current.data.getVehiclePlate().equals(vehiclePlate))	1	n	n
return current.data;	0	0	0
current=current.next;	1	n	n

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
return null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 3**		
BIG O	**O(n)**		

findByVehicleType()

CODE	S/E	FREQUENCY	TOTAL
//worst time complixity is o(n)	0	0	0
public LinkedList<IDriver> findByVehicleType(VehicleType vehicleType) {	0	0	0
LinkedList<IDriver>LVehicles=new LinkedList<>();	1	1	1
Node<IDriver>current=head;	1	1	1
while(current!=null) {	1	n+1	n+1
if(current.data.getVehicleType()==vehicleType)	1	n	n
LVehicles.insert(current.data);	1	n	n
current=current.next;	1	n	n
}	0	0	0
return LVehicles;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 4**		
BIG O	**O(n)**		

getAll()

CODE	S/E	FREQUENCY	TOTAL
//worst time complixity is o(n)	0	0	0
public LinkedList<IDriver> getAll() {	0	0	0
LinkedList<IDriver>ListOfAll=new LinkedList<>();	1	1	1
Node<IDriver>current =head;	1	1	1
while(current!=null)	1	n+1	n+1
{	0	0	0
ListOfAll.insert(current.data);	1	n	n
current=current.next;	1	n	n

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
return ListOfAll;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 4**		
BIG O	**O(n)**		

removeById()

CODE	S/E	FREQUENCY	TOTAL
//worst time complixity is o(n)	0	0	0
public boolean removeById(int driverId) {	0	0	0
if(head==null)	1	1	1
return false;	1	1	1
if(head.data.getId()==driverId) {	1	1	1
head=head.next;	0	0	0
size--;	0	0	0
return true;	0	0	0
}	0	0	0
Node<IDriver>current=head;	1	1	1
Node<IDriver>prev=null;	1	1	1
while(current!=null)	1	n	n
{	0	0	0
if(current.data.getId()==driverId)	1	n-1	n-1
{	0	0	0
prev.next=current.next;	0	0	0
size--;	0	0	0
return true;	0	0	0
}	0	0	0
prev=current;	1	n-1	n-1
current=current.next;	1	n-1	n-1
}	0	0	0
return false;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 3**		
BIG O	**O(n)**		

removeByVehiclePlate()

CODE	S/E	FREQUENCY	TOTAL
public boolean removeByVehiclePlate(String vehiclePlate) {	0	0	0
if(head==null)	1	1	1
return false;	1	1	1
if(head.data.getVehiclePlate().equals(vehiclePlate)) {	1	1	1
head=head.next;	0	0	0
size--;	0	0	0
return true;	0	0	0
}	0	0	0
Node<IDriver>current=head;	1	1	1
Node<IDriver>prev=null;	1	1	1
while(current!=null)	1	n	n
{	0	0	0
if(current.data.getVehiclePlate().equals(vehiclePlate))	1	n-1	n-1
{	0	0	0
prev.next=current.next;	0	0	0
size--;	0	0	0
return true;	0	0	0
}	0	0	0
prev=current;	1	n-1	n-1
current=current.next;	1	n-1	n-1
}	0	0	0
return false;	1	1	1
}//o(n) complixity	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 3**		
BIG O	**O(n)**		

removeByName()

CODE	S/E	FREQUENCY	TOTAL
public int removeByName(String fullName) {	0	0	0
if(head==null)	1	1	1
return 0;	0	0	0
Node<IDriver>current=head;	1	1	1
int countRemovedNames=0;	1	1	1
// i started from current.next to have reference point	0	0	0
Node<IDriver>prev=current;	1	1	1
current=current.next;	1	n-1	n-1
while(current!=null)	1	n	n
{	0	0	0
if(current.data.getName().equals(fullName)) {	1	n-1	n-1
prev.next=current.next;	1	n-1	n-1
countRemovedNames++;	1	1	1
size--;	1	1	1
}	0	0	0
else	0	0	0
prev=current;	0	0	0
current=current.next;	1	n-1	n-1
}	0	0	0
if(head.data.getName().equals(fullName)) {	1	1	1
head=head.next;	1	1	1
countRemovedNames++;	1	1	1
size--;	1	1	1
}	0	0	0
return countRemovedNames;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 5*n + 7**		
BIG O	**O(n)**		

removeByVehicleType()

CODE	S/E	FREQUENCY	TOTAL
// same logic is applied with method remvoeByName	0	0	0
//o(n) complixity	0	0	0
public int removeByVehicleType(VehicleType vehicleType) {	0	0	0
int countRemoveVechilces=0;	1	1	1
Node<IDriver>current=head;	1	1	1
if(head==null)	1	1	1
return 0;	0	0	0
Node<IDriver>prev=current;	1	1	1
current=current.next;	1	n-1	n-1
while(current!=null)	1	n	n
{	0	0	0
if(current.data.getVehicleType()==(vehicleType)) {	1	n-1	n-1
prev.next=current.next;	1	n-1	n-1
countRemoveVechilces++;	1	1	1
size--;	1	1	1
}	0	0	0
else	0	0	0
prev=current;	0	0	0
current=current.next;	1	n-1	n-1
}	0	0	0
if(head.data.getVehicleType()==(vehicleType)) {	1	1	1
head=head.next;	1	1	1
countRemoveVechilces++;	1	1	1
size--;	1	1	1
}	0	0	0
return countRemoveVechilces;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 5*n + 7**		
BIG O	**O(n)**		

size()

CODE	S/E	FREQUENCY	TOTAL
//o(1) complixity	0	0	0
public int size() {	0	0	0
return size;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

DateTime

DateTime (constructor)

CODE	S/E	FREQUENCY	TOTAL
//Student Name: نواف تركي القصير	0	0	0
//Student Number: 446107045	0	0	0
public class DateTime implements IDateTime {	0	0	0
private final int year;	0	0	0
private final int month;	0	0	0
private final int day;	0	0	0
private final int hour;	0	0	0
private final int minute;	0	0	0
//o(1) time compixity	0	0	0
public DateTime(int year, int month, int day, int hour, int minute) {	0	0	0
if(year <1)	1	1	1
throw new IllegalArgumentException("invalid year");	0	0	0
if (month < 1 || month > 12)	1	1	1
throw new IllegalArgumentException("invalid Month");	0	0	0
if(day<1 || day >31)	1	1	1
throw new IllegalArgumentException("invalid day");	0	0	0
if(hour < 0 || hour > 23)	1	1	1
throw new IllegalArgumentException("invalid hour");	0	0	0
if (minute < 0 || minute > 59)	1	1	1
throw new IllegalArgumentException("invalid minute");	0	0	0
this.year = year;	1	1	1
this.month = month;	1	1	1
this.day = day;	1	1	1

CODE	S/E	FREQUENCY	TOTAL
this.hour = hour;	1	1	1
this.minute = minute;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 10**		
BIG O	**O(1)**		

getYear()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time compixity	0	0	0
public int getYear() {	0	0	0
return year;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getMonth()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time compixity	0	0	0
public int getMonth() {	0	0	0
return month;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getDay()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time compixity	0	0	0
public int getDay() {	0	0	0
return day;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getHour()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time compixity	0	0	0
public int getHour() {	0	0	0
return hour;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getMinute()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time compixity	0	0	0
public int getMinute() {	0	0	0
return minute;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

format()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time compixity	0	0	0
public String format() {	0	0	0
String s= FormatDigits(month)+"/"+FormatDigits(day)+"/"+yearFormat(year)+	1	1	1
" "+FormatDigits(hour)+":"+FormatDigits(minute);	1	1	1
return s;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3**		
BIG O	**O(1)**		

FormatDigits()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time compixity	0	0	0
private String FormatDigits(int n) {	0	0	0
if(n<10) // if day =3 for instance it will be 03 to make the alignment works	1	1	1
return "0"+n;	0	0	0
else	0	0	0
return ""+n;	0	0	0
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

yearFormat()

CODE	S/E	FREQUENCY	TOTAL
//o(1) complixty	0	0	0
//year logic differs than month/day and time a year can have mroe than 2 digits	0	0	0
private String yearFormat(int year) {	0	0	0
if(year < 10)	1	1	1
return "000"+year;	1	1	1
else	0	0	0
if(year<100)	1	1	1
return "00"+year;	1	1	1
else	0	0	0

CODE	S/E	FREQUENCY	TOTAL
if(year<1000)	1	1	1
return "0"+year;	1	1	1
else	0	0	0
return ""+year;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 7**		
BIG O	**O(1)**		

parse()

CODE	S/E	FREQUENCY	TOTAL
//o(1) time compilixty	0	0	0
public static DateTime parse(String text) {	0	0	0
if (text == null)	1	1	1
throw new IllegalArgumentException("Invalid date");	0	0	0
String[] s = text.trim().split(" ");// here trim it remove the extra spaces	1	1	1
// split i going to split into to elemnts e0=date and e1=time	0	0	0
if (s.length != 2)	1	1	1
throw new IllegalArgumentException("Invalid date");	0	0	0
String[] date = s[0].split("/");	1	1	1
String[] time = s[1].split(":");	1	1	1
if (date.length != 3 || time.length != 2)	1	1	1
throw new IllegalArgumentException("Invalid date");	0	0	0
int month = Integer.parseInt(date[0]);	1	1	1
int day = Integer.parseInt(date[1]);	1	1	1
int year = Integer.parseInt(date[2]);	1	1	1
int hour = Integer.parseInt(time[0]);	1	1	1
int minute = Integer.parseInt(time[1]);	1	1	1
return new DateTime(year, month, day, hour, minute);	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 12**		
BIG O	**O(L)**		

compareTo()

CODE	S/E	FREQUENCY	TOTAL
public int compareTo(IDateTime other) {	0	0	0
if (year != other.getYear())	1	1	1
return year - other.getYear();	0	0	0
if (month != other.getMonth())	1	1	1
return month - other.getMonth();	0	0	0
if (day != other.getDay())	1	1	1
return day - other.getDay();	0	0	0
if (hour != other.getHour())	1	1	1
return hour - other.getHour();	0	0	0
return minute - other.getMinute();	0	0	0
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4**		
BIG O	**O(1)**		

toString()

CODE	S/E	FREQUENCY	TOTAL
public String toString() {	0	0	0
return format();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

Analysis — بسام فيصل الزهراني

Main

main()

CODE	S/E	FREQUENCY	TOTAL
public static void main(String[] args){	0	0	0
Scanner input = new Scanner(System.in);	1	1	1
RideSharingSystem system = new RideSharingSystem();	1	1	1
// Load the CSV files	0	0	0
boolean ridersLoaded = system.loadRidersFromCSV("riders_100.csv");	1	1	1
boolean driversLoaded = system.loadDriversFromCSV("drivers_30.csv");	1	1	1
boolean ridesLoaded = system.loadRidesFromCSV("rides_40.csv");	1	1	1
// Check loading results	0	0	0
if (ridersLoaded && driversLoaded && ridesLoaded)	1	1	1
System.out.println("All files loaded successfully.");	1	1	1
else	0	0	0
System.out.println("Some files failed to load. Data may be incomplete.");	1	1	1
int choice = -1;	1	1	1
while (choice != 0) {	1	n+1	n+1
// Show the menu	0	0	0
printMenu();	1	n	n
System.out.print("Enter your choice: ");	1	n	n
try {	0	0	0
choice = Integer.parseInt(input.nextLine());	1	n	n
switch (choice) {	1	n	n
case 1:	0	0	0
// List all riders	0	0	0
printList(system.getAllRiders());	1	n	n
break;	1	n	n
case 2:	0	0	0
// Search riders by city	0	0	0
System.out.print("Enter home city: ");	1	n	n
String city = input.nextLine();	1	n	n
printList(system.searchRidersByHomeCity(city));	1	n	n
break;	1	n	n
case 3:	0	0	0
// Search riders by name	0	0	0

CODE	S/E	FREQUENCY	TOTAL
System.out.print("Enter rider name: ");	1	n	n
String name = input.nextLine();	1	n	n
printList(system.searchRidersByName(name));	1	n	n
break;	1	n	n
case 4:	0	0	0
// Add a rider	0	0	0
System.out.print("Enter rider ID: ");	1	n	n
int riderId = Integer.parseInt(input.nextLine());	1	n	n
System.out.print("Enter name: ");	1	n	n
String riderName = input.nextLine();	1	n	n
System.out.print("Enter email: ");	1	n	n
String email = input.nextLine();	1	n	n
System.out.print("Enter phone number: ");	1	n	n
String riderPhone = input.nextLine();	1	n	n
System.out.print("Enter home city: ");	1	n	n
String homeCity = input.nextLine();	1	n	n
Rider rider = new Rider(riderId, riderName,	0	0	0
email, riderPhone, homeCity);	1	n	n
if (system.addRider(rider))	1	n	n
System.out.println("Rider added successfully.");	1	n	n
else	0	0	0
System.out.println("Failed to add rider.");	1	n	n
break;	1	n	n
case 5:	0	0	0
// List all drivers	0	0	0
printList(system.getAllDrivers());	1	n	n
break;	1	n	n
case 6:	0	0	0
// Search drivers by vehicle type	0	0	0
System.out.println("SEDAN, LUXURY_SEDAN, SUV, VAN");	1	n	n
System.out.print("Enter vehicle type: ");	1	n	n
VehicleType type = VehicleType.valueOf(input.nextLine());	1	n	n
printList(system.searchDriversByVehicleType(type));	1	n	n
break;	1	n	n
case 7:	0	0	0
// Add a driver	0	0	0
System.out.print("Enter driver ID: ");	1	n	n

CODE	S/E	FREQUENCY	TOTAL
int driverId = Integer.parseInt(input.nextLine());	1	n	n
System.out.print("Enter name: ");	1	n	n
String driverName = input.nextLine();	1	n	n
System.out.print("Enter phone number: ");	1	n	n
String driverPhone = input.nextLine();	1	n	n
System.out.print("Enter vehicle plate: ");	1	n	n
String plate = input.nextLine();	1	n	n
System.out.println("SEDAN, LUXURY_SEDAN, SUV, VAN");	1	n	n
System.out.print("Enter vehicle type: ");	1	n	n
VehicleType driverType =	0	0	0
VehicleType.valueOf(input.nextLine());	1	n	n
Driver driver = new Driver(driverId, driverName, driverPhone, plate, driverType);	1	n	n
if (system.addDriver(driver))	1	n	n
System.out.println("Driver added successfully.");	1	n	n
else	0	0	0
System.out.println("Failed to add driver.");	1	n	n
break;	1	n	n
case 8:	0	0	0
// List all rides	0	0	0
printList(system.getAllRidesAlphabetically());	1	n	n
break;	1	n	n
case 9:	0	0	0
// Search rides by pickup location	0	0	0
System.out.print("Enter pickup location: ");	1	n	n
String location = input.nextLine();	1	n	n
printList(system.searchRidesByPickupLocation(location));	1	n	n
break;	1	n	n
case 10:	0	0	0
// Search rides by rider name	0	0	0
System.out.print("Enter rider name: ");	1	n	n
String searchName = input.nextLine();	1	n	n
printList(system.searchRidesByRiderName(searchName));	1	n	n
break;	1	n	n
case 11:	0	0	0
// List shared ride participants	0	0	0
System.out.print("Enter pickup location: ");	1	n	n
String pickup = input.nextLine();	1	n	n

CODE	S/E	FREQUENCY	TOTAL
printList(system.getSharedRideParticipants(pickup));	1	n	n
break;	1	n	n
case 12:	0	0	0
// Add a ride	0	0	0
System.out.print("Enter ride type (PRIVATE/SHARED): ");	1	n	n
String rideType = input.nextLine();	1	n	n
if (!rideType.equals("PRIVATE") && !rideType.equals("SHARED")) {	1	n	n
System.out.println("Invalid ride type.");	1	n	n
break;	1	n	n
}	0	0	0
System.out.print("Enter pickup location: ");	1	n	n
String pickupLocation = input.nextLine();	1	n	n
System.out.print("Enter pickup time (MM/DD/YYYY HH:MM): ");	1	n	n
IDateTime pickupTime = readDateTime(input.nextLine());	1	n	n
System.out.print("Enter dropoff time (MM/DD/YYYY HH:MM): ");	1	n	n
IDateTime dropoffTime = readDateTime(input.nextLine());	1	n	n
System.out.print("Enter dropoff location: ");	1	n	n
String dropoffLocation = input.nextLine();	1	n	n
System.out.print("Enter driver ID: ");	1	n	n
int rideDriverId = Integer.parseInt(input.nextLine());	1	n	n
boolean added = false;	1	n	n
if (rideType.equals("PRIVATE")) {	1	n	n
System.out.print("Enter rider ID: ");	1	n	n
int privateRiderId = Integer.parseInt(input.nextLine());	1	n	n
added = system.schedulePrivateRide(pickupLocation, pickupTime, dropoffTime, dropoffLocation, privateRiderId, rideDriverId);	1	n	n
}	0	0	0
else {	1	n	n
System.out.print("Enter rider IDs separated by ; : ");	1	n	n
String[] ids = input.nextLine().split(";", -1);	1	n	n
if (ids.length < 2) {	1	n	n
System.out.println("Shared ride needs multiple riders.");	1	n	n
break;	1	n	n
}	0	0	0
int[] riderIds = new int[ids.length];	1	n	n
for (int i = 0; i < ids.length; i++) {	1	n(n+1)	n(n+1)
riderIds[i] = Integer.parseInt(ids[i]);	1	n^2	n^2
}	0	0	0

CODE	S/E	FREQUENCY	TOTAL
added = system.scheduleSharedRide(pickupLocation, pickupTime, dropoffTime, dropoffLocation, riderIds, rideDriverId);	1	n	n
}	0	0	0
if (added)	1	n	n
System.out.println("Ride added successfully.");	1	n	n
else	0	0	0
System.out.println("Failed to add ride.");	1	n	n
break;	1	n	n
case 0:	0	0	0
// Exit	0	0	0
System.out.println("Goodbye!");	1	n	n
break;	1	n	n
default:	0	0	0
System.out.println("Invalid option. Try again.");	1	n	n
}	0	0	0
} catch (IllegalArgumentException e) {	1	n	n
if(e.getMessage() !=null){	1	n	n
System.out.println("Invalid Input :" + e.getMessage());	1	n	n
}	0	0	0
// Handle invalid input	0	0	0
System.out.println("Invalid input. Try again.");	1	n	n
}	0	0	0
}	0	0	0
input.close();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2*n^2 + 107*n + 11**		
BIG O	**O(n^2)**		

printMenu()

CODE	S/E	FREQUENCY	TOTAL
public static void printMenu() {	0	0	0
System.out.println("\n===== Ride Sharing System =====");	1	1	1
System.out.println("1. List all riders.");	1	1	1
System.out.println("2. Search for riders by home city.");	1	1	1
System.out.println("3. Search for riders by name.");	1	1	1
System.out.println("4. Add a rider.");	1	1	1

CODE	S/E	FREQUENCY	TOTAL
System.out.println("5. List all drivers.");	1	1	1
System.out.println("6. Search for drivers by vehicle type.");	1	1	1
System.out.println("7. Add a driver.");	1	1	1
System.out.println("8. List all rides alphabetically by pickup location.");	1	1	1
System.out.println("9. Search for rides by pickup location.");	1	1	1
System.out.println("10. Search for rides by rider name.");	1	1	1
System.out.println("11. List the participants of a shared ride by pickup location.");	1	1	1
System.out.println("12. Add a ride.");	1	1	1
System.out.println("0. Exit the program.");	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 14**		
BIG O	**O(1)**		

printList()

CODE	S/E	FREQUENCY	TOTAL
public static <T> void printList(LinkedList<T> list) {	0	0	0
if (list == null || list.empty()) {	1	1	1
System.out.println("No results found.");	1	1	1
return;	1	1	1
}	0	0	0
list.findFirst();	1	1	1
while (!list.last()) {	1	n+1	n+1
System.out.println(list.retrieve().toString());	1	n	n
list.findNext();	1	n	n
}	0	0	0
// Print the last element	0	0	0
System.out.println(list.retrieve().toString());	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 6**		
BIG O	**O(n)**		

readDateTime()

CODE	S/E	FREQUENCY	TOTAL
public static IDateTime readDateTime(String text) {	0	0	0
String[] parts = text.split(" ", -1);	1	1	1
if (parts.length != 2)	1	1	1
throw new IllegalArgumentException();	1	1	1
String[] date = parts[0].split("/", -1);	1	1	1
String[] time = parts[1].split(":", -1);	1	1	1
if (date.length != 3 || time.length != 2)	1	1	1
throw new IllegalArgumentException();	1	1	1
int month = Integer.parseInt(date[0]);	1	1	1
int day = Integer.parseInt(date[1]);	1	1	1
int year = Integer.parseInt(date[2]);	1	1	1
int hour = Integer.parseInt(time[0]);	1	1	1
int minute = Integer.parseInt(time[1]);	1	1	1
return new DateTime(year, month, day, hour, minute);	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 13**		
BIG O	**O(1)**		

RideSharingSystem

RideSharingSystem (constructor)

CODE	S/E	FREQUENCY	TOTAL
public RideSharingSystem(){	0	0	0
drivers = new DriverList();	1	1	1
riders = new RiderList();	1	1	1
rides = new RideList();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3**		
BIG O	**O(1)**		

searchDriverById()

CODE	S/E	FREQUENCY	TOTAL
public IDriver searchDriverById(int driverId){	0	0	0
return drivers.findById(driverId);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

searchDriverByVehiclePlate()

CODE	S/E	FREQUENCY	TOTAL
public IDriver searchDriverByVehiclePlate(String vehiclePlate){	0	0	0
return drivers.findByVehiclePlate(vehiclePlate);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

getAllDrivers()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IDriver> getAllDrivers(){	0	0	0
return drivers.getAll();	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

searchDriversByVehicleType()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IDriver> searchDriversByVehicleType(VehicleType vehicleType){	0	0	0
return drivers.findByVehicleType(vehicleType);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

addDriver()

CODE	S/E	FREQUENCY	TOTAL
public boolean addDriver(IDriver driver){	0	0	0
return drivers.add(driver);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

searchRiderById()

CODE	S/E	FREQUENCY	TOTAL
public IRider searchRiderById(int riderId){	0	0	0
return riders.findById(riderId);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

searchRiderByEmail()

CODE	S/E	FREQUENCY	TOTAL
public IRider searchRiderByEmail(String email){	0	0	0
return riders.findByEmail(email);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

searchRidersByName()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRider> searchRidersByName(String fullName){	0	0	0
return riders.findByName(fullName);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

searchRidersByHomeCity()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRider> searchRidersByHomeCity(String homeCity){	0	0	0
return riders.findByHomeCity(homeCity);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

getAllRiders()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRider> getAllRiders(){	0	0	0
return riders.getAll();	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

addRider()

CODE	S/E	FREQUENCY	TOTAL
public boolean addRider(IRider rider){	0	0	0
if (rider == null)	1	1	1
return false;	1	0	0
else return riders.add(rider);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n + 1**		
BIG O	**O(n)**		

searchRidesByPickupLocation()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRide> searchRidesByPickupLocation(String pickupLocation){	0	0	0
return rides.findByPickupLocation(pickupLocation);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

searchRidesByRiderName()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRide> searchRidesByRiderName(String riderName){	0	0	0
return rides.findByRiderName(riderName);	n^2	1	n^2
}	0	0	0
TOTAL OPERATIONS	**$T(n) = n^2$**		
BIG O	**$O(n^2)$**		

getAllRidesAlphabetically()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRide> getAllRidesAlphabetically(){	0	0	0
return rides.getAllAlphabetically();	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

getSharedRideParticipants()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRider> getSharedRideParticipants(String pickupLocation){	0	0	0
LinkedList<IRide> sameRide = rides.findByPickupLocation(pickupLocation);//create a list with rides already stored with the pickuplocation	1	1	1
LinkedList<IRider> tmp = new LinkedList<>();	1	1	1
if (sameRide.empty())	1	1	1
return tmp;	1	1	1
sameRide.findFirst();//current = head	1	1	1
while(!sameRide.last()){	1	n+1	n+1
if(sameRide.retrieve() instanceof SharedRide sharedRide){//checks if the current is a sharedRide or privateRide	1	n	n
LinkedList<IRider> a = sharedRide.getParticipants();	1	n	n
if (!a.empty()){	1	n	n
a.findFirst();	1	n	n
while (!a.last()) {	1	n(n+1)	n(n+1)
tmp.insert(a.retrieve());	1	n^2	n^2
a.findNext();	1	n^2	n^2
}	0	0	0
tmp.insert(a.retrieve());	1	n	n

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
}	0	0	0
sameRide.findNext();	1	n	n
}//last check for sameRide	0	0	0
if(sameRide.retrieve() instanceof SharedRide sharedRide){	1	1	1
LinkedList<IRider> a = sharedRide.getParticipants();	1	1	1
if (!a.empty()){	1	1	1
a.findFirst();	1	1	1
while (!a.last()) {	1	n+1	n+1
tmp.insert(a.retrieve());	1	n	n
a.findNext();	1	n	n
}	0	0	0
tmp.insert(a.retrieve());	1	1	1
}	0	0	0
}	0	0	0
return tmp;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n^2 + 11*n + 13**		
BIG O	**O(n^2)**		

loadDriversFromCSV()

CODE	S/E	FREQUENCY	TOTAL
public boolean loadDriversFromCSV(String driversFilePath){	0	0	0
Scanner input = null;//so i can use it outside of try block	1	1	1
try {	0	0	0
File file = new File(driversFilePath);//open the drivers file	1	1	1
input = new Scanner(file);	1	1	1
while(input.hasNextLine()){//read the file lines	1	n+1	n+1
String Line = input.nextLine();	1	n	n
String[] data = Line.split(",", -1);	1	n	n
if (data.length != 5){//check its field	1	n	n
return false;	1	n	n
}	0	0	0
//get driver info	0	0	0
int id = Integer.parseInt(data[0].trim());//change string to int	1	n	n

CODE	S/E	FREQUENCY	TOTAL
String name = data[1];	1	n	n
String phone = data[2];	1	n	n
String plate = data[3];	1	n	n
VehicleType type = VehicleType.valueOf(data[4]);	1	n	n
Driver driver = new Driver(id, name, phone, plate, type);	1	n	n
if (!addDriver(driver))	1	n	n
return false;	1	n	n
}	0	0	0
if (input.ioException() != null)//check read errors	1	1	1
return false;	1	1	1
return true;	1	1	1
} catch (FileNotFoundException e) {	1	1	1
return false;	1	1	1
} catch (IllegalArgumentException e){	1	1	1
return false;	1	1	1
} finally {//close file	1	1	1
if (input !=null)	1	1	1
input.close();	1	1	1
}	0	0	0
}	0	0	0
TOTAL OPERATIONS	**T(n) = 13*n + 14**		
BIG O	**O(n)**		

loadRidersFromCSV()

CODE	S/E	FREQUENCY	TOTAL
public boolean loadRidersFromCSV(String ridersFilePath){	0	0	0
if(ridersFilePath == null)	1	1	1
return false;	1	1	1
Scanner input = null;//so i can use it outside of try block	1	1	1
try {	0	0	0
File file = new File(ridersFilePath);//open the riders file	1	1	1
input = new Scanner(file);	1	1	1
while(input.hasNextLine()){//read the file lines	1	n+1	n+1
String Line = input.nextLine();	1	n	n
String[] data = Line.split(",", -1);	1	n	n

CODE	S/E	FREQUENCY	TOTAL
if (data.length != 5){//check its field	1	n	n
return false;	1	n	n
}	0	0	0
//get rider info	0	0	0
int id = Integer.parseInt(data[0]);//change string to int	1	n	n
String name = data[1];	1	n	n
String email = data[2];	1	n	n
String phone = data[3];	1	n	n
String city = data[4];	1	n	n
Rider rider = new Rider(id, name, email, phone, city);	1	n	n
if (!addRider(rider))	1	n	n
return false;	1	n	n
}	0	0	0
if (input.ioException() != null)//check read errors	1	1	1
return false;	1	1	1
return true;	1	1	1
} catch (FileNotFoundException e) {	1	1	1
return false;	1	1	1
} catch (IllegalArgumentException e){	1	1	1
return false;	1	1	1
} finally {//close file	1	1	1
if (input !=null)	1	1	1
input.close();	1	1	1
}	0	0	0
}	0	0	0
TOTAL OPERATIONS	**T(n) = 13*n + 16**		
BIG O	**O(n)**		

removeDriver()

CODE	S/E	FREQUENCY	TOTAL
public boolean removeDriver(int driverId){	0	0	0
if (drivers.findById(driverId) == null){//check if theres driver	1	1	1
return false;	1	1	1
}	0	0	0
LinkedList<IRide> allRides = rides.getAllAlphabetically();	1	1	1

CODE	S/E	FREQUENCY	TOTAL
if (!allRides.empty()){	1	1	1
allRides.findFirst();	1	1	1
while(!allRides.last()){	1	n+1	n+1
if(allRides.retrieve().getDriver() != null && allRides.retrieve().getDriver().getId() == driverId){	1	n	n
rides.removeRideById(allRides.retrieve().getRideId());//remove driver ride	1	n	n
}	0	0	0
allRides.findNext();	1	n	n
}	0	0	0
if(allRides.retrieve().getDriver() != null && allRides.retrieve().getDriver().getId() == driverId){	1	1	1
rides.removeRideById(allRides.retrieve().getRideId());	1	1	1
}	0	0	0
}	0	0	0
//remove driver	0	0	0
return drivers.removeById(driverId);	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 9**		
BIG O	**O(n)**		

removeRider()

CODE	S/E	FREQUENCY	TOTAL
public boolean removeRider(int riderId) {	0	0	0
// Check rider exists	0	0	0
if (riders.findById(riderId) == null)	1	1	1
return false;	1	1	1
LinkedList<IRide> allRides = rides.getAllAlphabetically();	1	1	1
if (!allRides.empty()) {	1	1	1
allRides.findFirst();	1	1	1
while (!allRides.last()) {	1	n+1	n+1
IRide ride = allRides.retrieve();	1	n	n
// Check shared rides	0	0	0
if (ride instanceof SharedRide sharedRide) {	1	n	n
if (sharedRide.hasRider(riderId)) {	1	n	n
sharedRide.removeParticipantById(riderId);	1	n	n
if (sharedRide.isEmpty())	1	n	n
rides.removeRideById(ride.getRideId());	1	n	n

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
}	0	0	0
// Check private rides	0	0	0
else if (ride instanceof PrivateRide privateRide) {	1	n	n
if (privateRide.hasRider(riderId))	1	n	n
rides.removeRideById(ride.getRideId());	1	n	n
}	0	0	0
allRides.findNext();	1	n	n
}	0	0	0
// Check the last ride	0	0	0
IRide ride = allRides.retrieve();	1	1	1
if (ride instanceof SharedRide sharedRide) {	1	1	1
if (sharedRide.hasRider(riderId)) {	1	1	1
sharedRide.removeParticipantById(riderId);	1	1	1
if (sharedRide.isEmpty())	1	1	1
rides.removeRideById(ride.getRideId());	1	1	1
}	0	0	0
}	0	0	0
else if (ride instanceof PrivateRide privateRide) {	1	1	1
if (privateRide.hasRider(riderId))	1	1	1
rides.removeRideById(ride.getRideId());	1	1	1
}	0	0	0
}	0	0	0
// Remove the rider	0	0	0
return riders.removeById(riderId);	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 11*n + 16**		
BIG O	**O(n)**		

schedulePrivateRide()

CODE	S/E	FREQUENCY	TOTAL
public boolean schedulePrivateRide(String pickupLocation,	0	0	0
IDateTime pickupTime, IDateTime dropoffTime,	0	0	0
String dropoffLocation, int riderId, int driverId) {	0	0	0
// Check if rider exists	0	0	0
IRider rider = riders.findById(riderId);	1	1	1
if (rider == null)	1	1	1
return false;	1	1	1
// Check if driver exists	0	0	0
IDriver driver = drivers.findById(driverId);	1	1	1
if (driver == null)	1	1	1
return false;	1	1	1
// Check the time	0	0	0
if (pickupTime == null || dropoffTime == null)	1	1	1
return false;	1	1	1
if (pickupLocation == null || dropoffLocation == null)	1	1	1
return false;	1	1	1
if (pickupTime.compareTo(dropoffTime) >= 0)	1	1	1
return false;	1	1	1
// Get all rides	0	0	0
LinkedList<IRide> allRides = rides.getAllAlphabetically();	1	1	1
int newId = 1;	1	1	1
if (!allRides.empty()) {	1	1	1
allRides.findFirst();	1	1	1
while (!allRides.last()) {	1	n+1	n+1
IRide ride = allRides.retrieve();	1	n	n
// Find a new ride ID	0	0	0
if (ride.getRideId() >= newId)	1	n	n
newId = ride.getRideId() + 1;	1	n	n
// Check rider and driver	0	0	0
boolean sameRider = ride.hasRider(riderId);	1	n	n
boolean sameDriver = ride.getDriver() != null &&	0	0	0
ride.getDriver().getId() == driverId;	1	n	n
// Check time conflict	0	0	0
if (sameRider || sameDriver) {	1	n	n
if (pickupTime.compareTo(ride.getDropoffTime()) < 0 && dropoffTime.compareTo(ride.getPickupTime()) > 0)	1	n	n
return false;	1	n	n

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
allRides.findNext();	1	n	n
}	0	0	0
// Check the last ride	0	0	0
IRide ride = allRides.retrieve();	1	1	1
if (ride.getRideId() >= newId)	1	1	1
newId = ride.getRideId() + 1;	1	1	1
boolean sameRider = ride.hasRider(riderId);	1	1	1
boolean sameDriver = ride.getDriver() != null &&	0	0	0
ride.getDriver().getId() == driverId;	1	1	1
if (sameRider || sameDriver) {	1	1	1
if (pickupTime.compareTo(ride.getDropoffTime()) < 0 &&	1	1	1
dropoffTime.compareTo(ride.getPickupTime()) > 0)	0	0	0
return false;	1	1	1
}	0	0	0
}	0	0	0
// Create the private ride	0	0	0
PrivateRide newRide = new PrivateRide(newId,	0	0	0
pickupLocation, pickupTime, dropoffTime,	0	0	0
dropoffLocation, rider);	1	1	1
// Assign the driver	0	0	0
newRide.setDriver(driver);	1	1	1
// Add the ride	0	0	0
return rides.addRide(newRide);	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 10*n + 28**		
BIG O	**O(n)**		

scheduleSharedRide()

CODE	S/E	FREQUENCY	TOTAL
public boolean scheduleSharedRide(String pickupLocation,	0	0	0
IDateTime pickupTime, IDateTime dropoffTime,	0	0	0
String dropoffLocation, int[] riderIds, int driverId) {	0	0	0
// Check if driver exists	0	0	0
IDriver driver = drivers.findById(driverId);	1	1	1

CODE	S/E	FREQUENCY	TOTAL
if (driver == null)	1	1	1
return false;	1	1	1
// Check the riders	0	0	0
if (riderIds == null || riderIds.length == 0)	1	1	1
return false;	1	1	1
IRider[] rideRiders = new IRider[riderIds.length];	1	1	1
for (int i = 0; i < riderIds.length; i++) {	1	n+1	n+1
rideRiders[i] = riders.findById(riderIds[i]);	1	n	n
if (rideRiders[i] == null)	1	n	n
return false;	1	n	n
// Check duplicate riders	0	0	0
for (int j = 0; j < i; j++) {	1	n(n+1)	n(n+1)
if (riderIds[i] == riderIds[j])	1	n^2	n^2
return false;	1	n^2	n^2
}	0	0	0
}	0	0	0
// Check the time	0	0	0
if (pickupTime == null || dropoffTime == null)	1	1	1
return false;	1	1	1
if (pickupLocation == null || dropoffLocation == null)	1	1	1
return false;	1	1	1
if (pickupTime.compareTo(dropoffTime) >= 0)	1	1	1
return false;	1	1	1
// Get all rides	0	0	0
LinkedList<IRide> allRides = rides.getAllAlphabetically();	1	1	1
int newId = 1;	1	1	1
if (!allRides.empty()) {	1	1	1
allRides.findFirst();	1	1	1
while (!allRides.last()) {	1	n+1	n+1
IRide ride = allRides.retrieve();	1	n	n
// Find a new ride ID	0	0	0
if (ride.getRideId() >= newId)	1	n	n
newId = ride.getRideId() + 1;	1	n	n
// Check the driver	0	0	0
boolean sameDriver = ride.getDriver() != null &&	0	0	0
ride.getDriver().getId() == driverId;	1	n	n
// Check all riders	0	0	0

CODE	S/E	FREQUENCY	TOTAL
boolean sameRider = false;	1	n	n
for (int i = 0; i < riderIds.length; i++) {	1	n(n+1)	n(n+1)
if (ride.hasRider(riderIds[i]))	1	n^2	n^2
sameRider = true;	1	n^2	n^2
}	0	0	0
// Check time conflict	0	0	0
if (sameDriver || sameRider) {	1	n	n
if (pickupTime.compareTo(ride.getDropoffTime()) < 0 &&	1	n	n
dropoffTime.compareTo(ride.getPickupTime()) > 0)	0	0	0
return false;	1	n	n
}	0	0	0
allRides.findNext();	1	n	n
}	0	0	0
// Check the last ride	0	0	0
IRide ride = allRides.retrieve();	1	1	1
if (ride.getRideId() >= newId)	1	1	1
newId = ride.getRideId() + 1;	1	1	1
boolean sameDriver = ride.getDriver() != null &&	0	0	0
ride.getDriver().getId() == driverId;	1	1	1
boolean sameRider = false;	1	1	1
for (int i = 0; i < riderIds.length; i++) {	1	n+1	n+1
if (ride.hasRider(riderIds[i]))	1	n	n
sameRider = true;	1	n	n
}	0	0	0
if (sameDriver || sameRider) {	1	1	1
if (pickupTime.compareTo(ride.getDropoffTime()) < 0 &&	1	1	1
dropoffTime.compareTo(ride.getPickupTime()) > 0)	0	0	0
return false;	1	1	1
}	0	0	0
}	0	0	0
// Create the shared ride	0	0	0
SharedRide newRide = new SharedRide(newId,	0	0	0
pickupLocation, pickupTime, dropoffTime,	0	0	0
dropoffLocation);	1	1	1
// Add the riders	0	0	0
for (int i = 0; i < rideRiders.length; i++) {	1	n+1	n+1
if (!newRide.addParticipant(rideRiders[i]))	1	n	n

CODE	S/E	FREQUENCY	TOTAL
return false;	1	n	n
}	0	0	0
// Assign the driver	0	0	0
newRide.setDriver(driver);	1	1	1
// Add the ride	0	0	0
return rides.addRide(newRide);	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 6*n^2 + 22*n + 31**		
BIG O	**O(n^2)**		

loadRidesFromCSV()

CODE	S/E	FREQUENCY	TOTAL
public boolean loadRidesFromCSV(String ridesFilePath) {	0	0	0
if (ridesFilePath == null)	1	1	1
return false;	1	1	1
Scanner input = null;	1	1	1
try {	0	0	0
// Open the rides file	0	0	0
File file = new File(ridesFilePath);	1	1	1
input = new Scanner(file);	1	1	1
// Read each line	0	0	0
while (input.hasNextLine()) {	1	n+1	n+1
String line = input.nextLine();	1	n	n
String[] data = line.split(",", -1);	1	n	n
// Check the number of fields	0	0	0
if (data.length != 7)	1	n	n
return false;	1	n	n
// Get ride information	0	0	0
String type = data[0];	1	n	n
String pickupLocation = data[1];	1	n	n
String dropoffLocation = data[4];	1	n	n
int driverId = Integer.parseInt(data[5]);	1	n	n
// Get pickup date and time	0	0	0
String[] pickup = data[2].split(" ", -1);	1	n	n
if (pickup.length != 2)	1	n	n

CODE	S/E	FREQUENCY	TOTAL
return false;	1	n	n
String[] pickupDate = pickup[0].split("/", -1);	1	n	n
String[] pickupClock = pickup[1].split(":", -1);	1	n	n
if (pickupDate.length != 3 || pickupClock.length != 2)	1	n	n
return false;	1	n	n
IDateTime pickupTime = new DateTime(	0	0	0
Integer.parseInt(pickupDate[2]),	0	0	0
Integer.parseInt(pickupDate[0]),	0	0	0
Integer.parseInt(pickupDate[1]),	0	0	0
Integer.parseInt(pickupClock[0]),	0	0	0
Integer.parseInt(pickupClock[1])	0	0	0
);	1	n	n
// Get dropoff date and time	0	0	0
String[] dropoff = data[3].split(" ", -1);	1	n	n
if (dropoff.length != 2)	1	n	n
return false;	1	n	n
String[] dropoffDate = dropoff[0].split("/", -1);	1	n	n
String[] dropoffClock = dropoff[1].split(":", -1);	1	n	n
if (dropoffDate.length != 3 || dropoffClock.length != 2)	1	n	n
return false;	1	n	n
IDateTime dropoffTime = new DateTime(	0	0	0
Integer.parseInt(dropoffDate[2]),	0	0	0
Integer.parseInt(dropoffDate[0]),	0	0	0
Integer.parseInt(dropoffDate[1]),	0	0	0
Integer.parseInt(dropoffClock[0]),	0	0	0
Integer.parseInt(dropoffClock[1])	0	0	0
);	1	n	n
// Check private ride	0	0	0
if (type.equals("PRIVATE")) {	1	n	n
int riderId = Integer.parseInt(data[6]);	1	n	n
if (!schedulePrivateRide(pickupLocation,	1	n	n
pickupTime, dropoffTime,	0	0	0
dropoffLocation, riderId, driverId))	0	0	0
return false;	1	n	n
}	0	0	0
// Check shared ride	0	0	0
else if (type.equals("SHARED")) {	1	n	n

CODE	S/E	FREQUENCY	TOTAL
String[] ids = data[6].split(";", -1);	1	n	n
if (ids.length < 2)	1	n	n
return false;	1	n	n
int[] riderIds = new int[ids.length];	1	n	n
for (int i = 0; i < ids.length; i++) {	1	n(n+1)	n(n+1)
riderIds[i] = Integer.parseInt(ids[i]);	1	n^2	n^2
}	0	0	0
if (!scheduleSharedRide(pickupLocation,	1	n	n
pickupTime, dropoffTime,	0	0	0
dropoffLocation, riderIds, driverId))	0	0	0
return false;	1	n	n
}	0	0	0
else {	1	n	n
return false;	1	n	n
}	0	0	0
}	0	0	0
// Check errors	0	0	0
if (input.ioException() != null)	1	1	1
return false;	1	1	1
return true;	1	1	1
} catch (FileNotFoundException e) {	1	1	1
return false;	1	1	1
} catch (IllegalArgumentException e) {	1	1	1
return false;	1	1	1
} finally {	1	1	1
// Close the file	0	0	0
if (input != null)	1	1	1
input.close();	1	1	1
}	0	0	0
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2*n^2 + 39*n + 16**		
BIG O	**O(n^2)**		

Analysis — وليد داني حلبي

LinkedList

LinkedList (constructor)

CODE	S/E	FREQUENCY	TOTAL
LINKED LIST	—	—	—
public LinkedList ()	0	0	0
{	0	0	0
head = current = tail = null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

empty()

CODE	S/E	FREQUENCY	TOTAL
public boolean empty ()	0	0	0
{	0	0	0
return head == null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

last()

CODE	S/E	FREQUENCY	TOTAL
public boolean last ()	0	0	0
{	0	0	0
return current == tail;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

full()

CODE	S/E	FREQUENCY	TOTAL
public boolean full ()	0	0	0
{	0	0	0
return false;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

findFirst()

CODE	S/E	FREQUENCY	TOTAL
public void findFirst ()	0	0	0
{	0	0	0
current = head;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

findNext()

CODE	S/E	FREQUENCY	TOTAL
public void findNext ()	0	0	0
{	0	0	0
if (current != null) current = current.next;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

findPrevious()

CODE	S/E	FREQUENCY	TOTAL
public void findPrevious ()	0	0	0
{	0	0	0
if (current != null) current = current.prev;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

findLast()

CODE	S/E	FREQUENCY	TOTAL
public void findLast ()	0	0	0
{	0	0	0
current = tail;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

findAt()

CODE	S/E	FREQUENCY	TOTAL
public void findAt(int index)	0	0	0
{	0	0	0
if (empty()) return;	1	1	1
if (index <= 0)	1	1	1
{	0	0	0
current = head;	1	0	0
return;	1	0	0
}	0	0	0
if (index >= size - 1)	1	1	1
{	0	0	0
current = tail;	1	0	0
return;	1	0	0
}	0	0	0
current = head;	1	1	1

CODE	S/E	FREQUENCY	TOTAL
for (int i = 0; i < index; i++)	1	n+1	n+1
{	0	0	0
current = current.next;	1	n	n
}	0	0	0
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2*n + 5**		
BIG O	**O(n)**		

retrieve()

CODE	S/E	FREQUENCY	TOTAL
public T retrieve ()	0	0	0
{	0	0	0
return current != null ? current.data : null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

update()

CODE	S/E	FREQUENCY	TOTAL
public void update (T data)	0	0	0
{	0	0	0
if (current != null) current.data = data;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

insertHead()

CODE	S/E	FREQUENCY	TOTAL
public void insertHead(T data)	0	0	0
{	0	0	0
Node<T> tmp = new Node<>(data);	1	1	1
if (empty())	1	1	1
{	0	0	0
head = tail = current = tmp;	1	0	0
}	0	0	0
else	0	0	0
{	0	0	0
tmp.next = head;	1	1	1
head.prev = tmp;	1	1	1
head = tmp;	1	1	1
}	0	0	0
size++;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 6**		
BIG O	**O(1)**		

insertTail()

CODE	S/E	FREQUENCY	TOTAL
public void insertTail(T data)	0	0	0
{	0	0	0
Node<T> tmp = new Node<>(data);	1	1	1
if (empty())	1	1	1
{	0	0	0
head = tail = current = tmp;	1	0	0
}	0	0	0
else	0	0	0
{	0	0	0
tail.next = tmp;	1	1	1
tmp.prev = tail;	1	1	1
tail = tmp;	1	1	1

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
size++;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 6**		
BIG O	**O(1)**		

insert()

CODE	S/E	FREQUENCY	TOTAL
public void insert(T data)	0	0	0
{	0	0	0
if (empty())	1	1	1
{	0	0	0
insertHead(data);	1	0	0
return;	1	0	0
}	0	0	0
Node<T> tmp = new Node<>(data);	1	1	1
tmp.next = current.next;	1	1	1
tmp.prev = current;	1	1	1
if (last())	1	1	1
tail = tmp;	1	1	1
else	0	0	0
current.next.prev = tmp;	1	1	1
current.next = tmp;	1	1	1
findNext();	1	1	1
size++;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 10**		
BIG O	**O(1)**		

insertAt()

CODE	S/E	FREQUENCY	TOTAL
public void insertAt(T data, int index)	0	0	0
{	0	0	0
if (empty() || index <= 0)	1	1	1
{	0	0	0
insertHead(data);	1	0	0
return;	1	0	0
}	0	0	0
if (index >= size)	1	1	1
{	0	0	0
insertTail(data);	1	0	0
return;	1	0	0
}	0	0	0
Node<T> tmp = head;	1	1	1
for (int i = 0; i < index - 1; i++)	1	n	n
{	0	0	0
tmp = tmp.next;	1	n-1	n-1
}	0	0	0
Node<T> newNode = new Node<>(data);	1	1	1
newNode.next = tmp.next;	1	1	1
newNode.prev = tmp;	1	1	1
tmp.next.prev = newNode;	1	1	1
tmp.next = newNode;	1	1	1
size++;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2*n + 8**		
BIG O	**O(n)**		

removeHead()

CODE	S/E	FREQUENCY	TOTAL
public void removeHead()	0	0	0
{	0	0	0
if (empty()) return;	1	1	1
if (head == tail)	1	1	1
{	0	0	0

CODE	S/E	FREQUENCY	TOTAL
head = tail = current = null;	1	0	0
}	0	0	0
else	0	0	0
{	0	0	0
if (current == head) current = head.next;	1	1	1
head = head.next;	1	1	1
head.prev = null;	1	1	1
}	0	0	0
size--;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 6**		
BIG O	**O(1)**		

removeTail()

CODE	S/E	FREQUENCY	TOTAL
public void removeTail()	0	0	0
{	0	0	0
if (empty()) return;	1	1	1
if (head == tail)	1	1	1
{	0	0	0
head = tail = current = null;	1	0	0
}	0	0	0
else	0	0	0
{	0	0	0
if (current == tail) current = tail.prev;	1	1	1
tail = tail.prev;	1	1	1
tail.next = null;	1	1	1
}	0	0	0
size--;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 6**		
BIG O	**O(1)**		

removeInfront()

CODE	S/E	FREQUENCY	TOTAL
public void removeInfront()	0	0	0
{	0	0	0
if (empty() || current == head) return;	1	1	1
if (current.prev == head)	1	1	1
{	0	0	0
removeHead();	1	0	0
return;	1	0	0
}	0	0	0
Node<T> toRemove = current.prev;	1	1	1
toRemove.prev.next = current;	1	1	1
current.prev = toRemove.prev;	1	1	1
size--;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 6**		
BIG O	**O(1)**		

removeBehind()

CODE	S/E	FREQUENCY	TOTAL
public void removeBehind()	0	0	0
{	0	0	0
if (empty() || current == tail) return;	1	1	1
if (current.next == tail)	1	1	1
{	0	0	0
removeTail();	1	0	0
return;	1	0	0
}	0	0	0
Node<T> toRemove = current.next;	1	1	1
current.next = toRemove.next;	1	1	1
toRemove.next.prev = current;	1	1	1
size--;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 6**		
BIG O	**O(1)**		

remove()

CODE	S/E	FREQUENCY	TOTAL
public void remove ()	0	0	0
{	0	0	0
if (empty()) return;	1	1	1
if (current == head)	1	1	1
{	0	0	0
removeHead();	1	0	0
return;	1	0	0
}	0	0	0
if (current == tail)	1	1	1
{	0	0	0
removeTail();	1	0	0
return;	1	0	0
}	0	0	0
current.prev.next = current.next;	1	1	1
current.next.prev = current.prev;	1	1	1
current = current.next;	1	1	1
size--;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 7**		
BIG O	**O(1)**		

removeAt()

CODE	S/E	FREQUENCY	TOTAL
public void removeAt(int index)	0	0	0
{	0	0	0
if (empty()) return;	1	1	1
if (index <= 0)	1	1	1
{	0	0	0
removeHead();	1	0	0
return;	1	0	0
}	0	0	0
if (index >= size - 1)	1	1	1

CODE	S/E	FREQUENCY	TOTAL

{
removeTail();
return;
}
Node<T> tmp = head;
for (int i = 0; i < index; i++)
{
tmp = tmp.next;
}
if (current == tmp)
{
current = tmp.next;
}
tmp.prev.next = tmp.next;
tmp.next.prev = tmp.prev;
size--;
}
	0
	1
	1
	0
	1
	1
	0
	1
	0
	1
	0
	1
	1
	1
	0
		0
		0
		0
		0
		1
		n+1
		0
		n
		0
		1
		0
		1
		0
		1
		0
			0
			0
			0
			0
			1
			n+1
			0
			n
			0
			1
			0
			1
			0
			1
			0

TOTAL OPERATIONS	**T(n) = 2*n + 10**		
BIG O	**O(n)**		

size()

CODE	S/E	FREQUENCY	TOTAL

public int size()
{
return size;
}
	0
	0
	1
	0
		0
		0
		1
		0
			0
			0
			1
			0

TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

Node

Node (constructor)

CODE	S/E	FREQUENCY	TOTAL
NODE	—	—	—
public Node()	0	0	0
{	0	0	0
this(null);	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

Node (constructor)

CODE	S/E	FREQUENCY	TOTAL
public Node(T data)	0	0	0
{	0	0	0
this.data = data;	1	1	1
next = null;	1	1	1
prev = null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3**		
BIG O	**O(1)**		

RiderList

add()

CODE	S/E	FREQUENCY	TOTAL
public boolean add(IRider rider)	0	0	0
{	0	0	0
//empty logic	0	0	0
if(riderList.empty())	1	1	1
{	0	0	0
riderList.insert(rider);	1	0	0
return true;	1	0	0

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
//existance check	0	0	0
if (findByEmail(rider.getEmail()) != null)	n	1	n
return false;	1	0	0
if (findById(rider.getId()) != null)	n	1	n
return false;	1	0	0
riderList.findFirst();	1	1	1
//head logic	0	0	0
if(rider.compareTo(riderList.retrieve()) < 0)	1	1	1
{	0	0	0
riderList.insertHead(rider);	1	0	0
return true;	1	0	0
}	0	0	0
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
riderList.findNext();	1	n-1	n-1
if (rider.compareTo(riderList.retrieve()) < 0)	1	n-1	n-1
{	0	0	0
riderList.findPrevious();	1	0	0
riderList.insert(rider);	1	0	0
return true;	1	0	0
}	0	0	0
}	0	0	0
//if its bigger than everything	0	0	0
riderList.insertTail(rider);	1	1	1
return true;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 5*n + 3**		
BIG O	**O(n)**		

findById()

CODE	S/E	FREQUENCY	TOTAL
public IRider findById(int riderId)	0	0	0
{	0	0	0
//empty logic	0	0	0
if (riderList.empty())	1	1	1
return null;	1	1	1
riderList.findFirst();	1	1	1
//head logic	0	0	0
if (riderList.retrieve().getId() == riderId)	1	1	1
return riderList.retrieve();	1	0	—
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
riderList.findNext();	1	n-1	n-1
if (riderList.retrieve().getId() == riderId)	1	n-1	n-1
return riderList.retrieve();	1	0	0
}	0	0	0
//found none	0	0	0
return null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 3**		
BIG O	**O(n)**		

findByName()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRider> findByName(String fullName)	0	0	0
{	0	0	0
LinkedList<IRider> list = new LinkedList<IRider>();	1	1	1
//empty logic	0	0	0
if (riderList.empty())	1	1	1
return list;	1	1	1
riderList.findFirst();	1	1	1
//head logic	0	0	0
if (riderList.retrieve().getName().equals(fullName))	1	1	1
list.insert(riderList.retrieve());	1	1	1

CODE	S/E	FREQUENCY	TOTAL
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
riderList.findNext();	1	n-1	n-1
if (riderList.retrieve().getName().equals(fullName))	1	n-1	n-1
list.insert(riderList.retrieve());	1	n-1	n-1
}	0	0	0
return list;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 4**		
BIG O	**O(n)**		

findByEmail()

CODE	S/E	FREQUENCY	TOTAL
public IRider findByEmail(String email)	0	0	0
{	0	0	0
//empty logic	0	0	0
if (riderList.empty())	1	1	1
return null;	1	1	1
riderList.findFirst();	1	1	1
//head logic	0	0	0
if (riderList.retrieve().getEmail().equals(email))	1	1	1
return riderList.retrieve();	1	0	0
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
riderList.findNext();	1	n-1	n-1
if (riderList.retrieve().getEmail().equals(email))	1	n-1	n-1
return riderList.retrieve();	1	0	0
}	0	0	0

CODE	S/E	FREQUENCY	TOTAL
//found none	0	0	0
return null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 3**		
BIG O	**O(n)**		

findByHomeCity()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRider> findByHomeCity(String homeCity)	0	0	0
{	0	0	0
LinkedList<IRider> list = new LinkedList<IRider>();	1	1	1
//empty logic	0	0	0
if (riderList.empty())	1	1	1
return list;	1	1	1
riderList.findFirst();	1	1	1
//head logic	0	0	0
if (riderList.retrieve().getHomeCity().equals(homeCity))	1	1	1
list.insert(riderList.retrieve());	1	1	1
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
riderList.findNext();	1	n-1	n-1
if (riderList.retrieve().getHomeCity().equals(homeCity))	1	n-1	n-1
list.insert(riderList.retrieve());	1	n-1	n-1
}	0	0	0
return list;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 4**		
BIG O	**O(n)**		

getAll()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRider> getAll()	0	0	0
{	0	0	0
LinkedList<IRider> list = new LinkedList<IRider>();	1	1	1
//empty logic	0	0	0
if (riderList.empty())	1	1	1
return list;	1	1	1
riderList.findFirst();	1	1	1
//head logic	0	0	0
list.insert(riderList.retrieve());	1	1	1
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
riderList.findNext();	1	n-1	n-1
list.insert(riderList.retrieve());	1	n-1	n-1
}	0	0	0
return list;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 4**		
BIG O	**O(n)**		

removeById()

CODE	S/E	FREQUENCY	TOTAL
public boolean removeById(int riderId)	0	0	0
{	0	0	0
//empty logic	0	0	0
if (riderList.empty())	1	1	1
return false;	1	1	1
riderList.findFirst();	1	1	1
//head logic	0	0	0
if (riderList.retrieve().getId() == riderId)	1	1	1
{	0	0	0
riderList.removeHead();	1	0	0
return true;	1	0	0
}	0	0	0

CODE	S/E	FREQUENCY	TOTAL
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
riderList.findNext();	1	n-1	n-1
if (riderList.retrieve().getId() == riderId)	1	n-1	n-1
{	0	0	0
riderList.remove();	1	0	0
return true;	1	0	0
}	0	0	0
}	0	0	0
//found none	0	0	0
return false;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 3**		
BIG O	**O(n)**		

removeByEmail()

CODE	S/E	FREQUENCY	TOTAL
public boolean removeByEmail(String email)	0	0	0
{	0	0	0
//empty logic	0	0	0
if (riderList.empty())	1	1	1
return false;	1	1	1
riderList.findFirst();	1	1	1
//head logic	0	0	0
if (riderList.retrieve().getEmail().equals(email))	1	1	1
{	0	0	0
riderList.removeHead();	1	0	0
return true;	1	0	0
}	0	0	0
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
riderList.findNext();	1	n-1	n-1
if (riderList.retrieve().getEmail().equals(email))	1	n-1	n-1

CODE	S/E	FREQUENCY	TOTAL
{	0	0	0
riderList.remove();	1	0	0
return true;	1	0	0
}	0	0	0
}	0	0	0
//found none	0	0	0
return false;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 3**		
BIG O	**O(n)**		

removeByName()

CODE	S/E	FREQUENCY	TOTAL
public int removeByName(String fullName)	0	0	0
{	0	0	0
//empty logic	0	0	0
if (riderList.empty())	1	1	1
return 0;	1	1	1
int count = 0;	1	1	1
riderList.findFirst();	1	1	1
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
if (riderList.retrieve().getName().equals(fullName))	1	n-1	n-1
{	0	0	0
riderList.remove();	1	n-1	n-1
count++;	1	n-1	n-1
}	0	0	0
else	0	0	0
riderList.findNext();	1	0	0
}	0	0	0
//tail logic	0	0	0
if (riderList.retrieve().getName().equals(fullName))	1	1	1
{	0	0	0
riderList.removeTail();	1	1	1

CODE	S/E	FREQUENCY	TOTAL
count++;	1	1	1
}	0	0	0
return count;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 5**		
BIG O	**O(n)**		

removeByHomeCity()

CODE	S/E	FREQUENCY	TOTAL
public int removeByHomeCity(String homeCity)	0	0	0
{	0	0	0
//empty logic	0	0	0
if (riderList.empty())	1	1	1
return 0;	1	1	1
int count = 0;	1	1	1
riderList.findFirst();	1	1	1
//normal logic	0	0	0
while (!riderList.last())	1	n	n
{	0	0	0
if (riderList.retrieve().getHomeCity().equals(homeCity))	1	n-1	n-1
{	0	0	0
riderList.remove();	1	n-1	n-1
count++;	1	n-1	n-1
}	0	0	0
else	0	0	0
riderList.findNext();	1	0	0
}	0	0	0
//tail logic	0	0	0
if (riderList.retrieve().getHomeCity().equals(homeCity))	1	1	1
{	0	0	0
riderList.removeTail();	1	1	1
count++;	1	1	1

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
return count;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 5**		
BIG O	**O(n)**		

size()

CODE	S/E	FREQUENCY	TOTAL
public int size()	0	0	0
{	0	0	0
return riderList.size();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

Rider

Rider (constructor)

CODE	S/E	FREQUENCY	TOTAL
public Rider(int id, String name, String email, String phoneNumber, String homeCity)	0	0	0
{	0	0	0
super(id, name, phoneNumber);	1	1	1
this.email = email;	1	1	1
this.homeCity = homeCity;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3**		
BIG O	**O(1)**		

getEmail()

CODE	S/E	FREQUENCY	TOTAL
public String getEmail()	0	0	0
{	0	0	0
return email;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setEmail()

CODE	S/E	FREQUENCY	TOTAL
public void setEmail(String email)	0	0	0
{	0	0	0
this.email = email;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getHomeCity()

CODE	S/E	FREQUENCY	TOTAL
public String getHomeCity()	0	0	0
{	0	0	0
return homeCity;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setHomeCity()

CODE	S/E	FREQUENCY	TOTAL
public void setHomeCity(String homeCity)	0	0	0
{	0	0	0
this.homeCity = homeCity;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

compareTo()

CODE	S/E	FREQUENCY	TOTAL
public int compareTo(IRider other)	0	0	0
{	0	0	0
int compare = getId() - other.getId();	1	1	1
return compare;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2**		
BIG O	**O(1)**		

RideList

addRide()

CODE	S/E	FREQUENCY	TOTAL
public boolean addRide(IRide ride)	0	0	0
{	0	0	0
//empty logic	0	0	0
if(rideList.empty())	1	1	1
{	0	0	0
rideList.insert(ride);	1	0	0
return true;	1	0	0
}	0	0	0
//existance check	0	0	0
if (findRideById(ride.getRideId()) != null)	n	1	n
return false;	1	0	0

CODE	S/E	FREQUENCY	TOTAL
rideList.findFirst();	1	1	1
//head logic	0	0	0
if(ride.compareTo(rideList.retrieve()) < 0)	1	1	1
{	0	0	0
rideList.insertHead(ride);	1	0	0
return true;	1	0	0
}	0	0	0
//normal logic	0	0	0
while (!rideList.last())	1	n	n
{	0	0	0
rideList.findNext();	1	n-1	n-1
if (ride.compareTo(rideList.retrieve()) < 0)	1	n-1	n-1
{	0	0	0
rideList.findPrevious();	1	0	0
rideList.insert(ride);	1	0	0
return true;	1	0	0
}	0	0	0
}	0	0	0
//last alphabet	0	0	0
rideList.insertTail(ride);	1	1	1
return true;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 3**		
BIG O	**O(n)**		

removeRideById()

CODE	S/E	FREQUENCY	TOTAL
public boolean removeRideById(int rideId)	0	0	0
{	0	0	0
//empty logic	0	0	0
if (rideList.empty())	1	1	1
return false;	1	1	1
rideList.findFirst();	1	1	1
//head logic	0	0	0
if (rideList.retrieve().getRideId() == rideId)	1	1	1

CODE	S/E	FREQUENCY	TOTAL
{	0	0	0
rideList.removeHead();	1	0	0
return true;	1	0	0
}	0	0	0
//normal logic	0	0	0
while (!rideList.last())	1	n	n
{	0	0	0
rideList.findNext();	1	n-1	n-1
if (rideList.retrieve().getRideId() == rideId)	1	n-1	n-1
{	0	0	0
rideList.remove();	1	0	0
return true;	1	0	0
}	0	0	0
}	0	0	0
//found none	0	0	0
return false;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 3**		
BIG O	**O(n)**		

getAllAlphabetically()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRide> getAllAlphabetically()	0	0	0
{	0	0	0
//its already alphabetical	0	0	0
LinkedList<IRide> list = new LinkedList<IRide>();	1	1	1
//empty logic	0	0	0
if (rideList.empty())	1	1	1
return list;	1	1	1
rideList.findFirst();	1	1	1
//head logic	0	0	0
list.insert(rideList.retrieve());	1	1	1
//normal logic	0	0	0
while (!rideList.last())	1	n	n
{	0	0	0

CODE	S/E	FREQUENCY	TOTAL
rideList.findNext();	1	n-1	n-1
list.insert(rideList.retrieve());	1	n-1	n-1
}	0	0	0
return list;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 4**		
BIG O	**O(n)**		

findByPickupLocation()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRide> findByPickupLocation(String pickupLocation)	0	0	0
{	0	0	0
LinkedList<IRide> list = new LinkedList<IRide>();	1	1	1
//empty logic	0	0	0
if (rideList.empty())	1	1	1
return list;	1	1	1
rideList.findFirst();	1	1	1
//head logic	0	0	0
if (rideList.retrieve().getPickupLocation().equals(pickupLocation))	1	1	1
list.insert(rideList.retrieve());	1	1	1
//normal logic	0	0	0
while (!rideList.last())	1	n	n
{	0	0	0
rideList.findNext();	1	n-1	n-1
if (rideList.retrieve().getPickupLocation().equals(pickupLocation))	1	n-1	n-1
list.insert(rideList.retrieve());	1	n-1	n-1
}	0	0	0
return list;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 4*n + 4**		
BIG O	**O(n)**		

findByRiderName()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRide> findByRiderName(String riderFullName)	0	0	0
{	0	0	0
LinkedList<IRide> list = new LinkedList<IRide>();	1	1	1
//empty logic	0	0	0
if (rideList.empty())	1	1	1
return list;	1	1	1
rideList.findFirst();	1	1	1
//head logic	0	0	0
if(rideList.retrieve() instanceof PrivateRide privateRide)//private ride	1	1	1
{	0	0	0
if(privateRide.hasRider(riderFullName))	1	0	0
list.insert(privateRide);	1	0	0
}	0	0	0
else if(rideList.retrieve() instanceof SharedRide sharedRide)//shared ride	1	1	1
{	0	0	0
if(sharedRide.hasRider(riderFullName))	n	1	n
list.insert(sharedRide);	n	1	n
}	0	0	0
//normal logic	0	0	0
while (!rideList.last())	1	n	n
{	0	0	0
rideList.findNext();	1	n-1	n-1
if(rideList.retrieve() instanceof PrivateRide privateRide)//private ride	1	n-1	n-1
{	0	0	0
if(privateRide.hasRider(riderFullName))	1	0	0
list.insert(privateRide);	1	0	0
}	0	0	0
else if(rideList.retrieve() instanceof SharedRide sharedRide)//shared ride	1	n-1	n-1
{	0	0	0
if(sharedRide.hasRider(riderFullName))	n	n-1	n(n-1)
list.insert(sharedRide);	n	n-1	n(n-1)
}	0	0	0

CODE	S/E	FREQUENCY	TOTAL
}	0	0	0
return list;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**$T(n) = 2n^2 + 4n + 4$**		
BIG O	**$O(n^2)$**		

size()

CODE	S/E	FREQUENCY	TOTAL
public int size()	0	0	0
{	0	0	0
return rideList.size();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

findRideById()

CODE	S/E	FREQUENCY	TOTAL
private IRide findRideById(int rideId)	0	0	0
{	0	0	0
//empty logic	0	0	0
if (rideList.empty())	1	1	1
return null;	1	1	1
rideList.findFirst();	1	1	1
//head logic	0	0	0
if (rideList.retrieve().getRideId() == rideId)	1	1	1
return rideList.retrieve();	1	0	0
//normal logic	0	0	0
while (!rideList.last())	1	n	n
{	0	0	0
rideList.findNext();	1	n-1	n-1
if (rideList.retrieve().getRideId() == rideId)	1	n-1	n-1
return rideList.retrieve();	1	0	0
}	0	0	0

CODE	S/E	FREQUENCY	TOTAL
//found none	0	0	0
return null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 3*n + 3**		
BIG O	**O(n)**		

Ride

Ride (constructor)

CODE	S/E	FREQUENCY	TOTAL
public Ride(int rideId, String pickupLocation, IDateTime pickupTime, IDateTime dropoffTime, String dropoffLocation)	0	0	0
{	0	0	0
this.rideId = rideId;	1	1	1
this.pickupLocation = pickupLocation;	1	1	1
this.pickupTime = pickupTime;	1	1	1
this.dropoffTime = dropoffTime;	1	1	1
this.dropoffLocation = dropoffLocation;	1	1	1
this.driver = null;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 6**		
BIG O	**O(1)**		

getRideId()

CODE	S/E	FREQUENCY	TOTAL
public int getRideId()	0	0	0
{	0	0	0
return rideId;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getPickupLocation()

CODE	S/E	FREQUENCY	TOTAL
public String getPickupLocation()	0	0	0
{	0	0	0
return pickupLocation;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setPickupLocation()

CODE	S/E	FREQUENCY	TOTAL
public void setPickupLocation(String pickupLocation)	0	0	0
{	0	0	0
this.pickupLocation = pickupLocation;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getPickupTime()

CODE	S/E	FREQUENCY	TOTAL
public IDateTime getPickupTime()	0	0	0
{	0	0	0
return pickupTime;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getDropoffTime()

CODE	S/E	FREQUENCY	TOTAL
public IDateTime getDropoffTime()	0	0	0
{	0	0	0
return dropoffTime;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getDropoffLocation()

CODE	S/E	FREQUENCY	TOTAL
public String getDropoffLocation()	0	0	0
{	0	0	0
return dropoffLocation;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setDropoffLocation()

CODE	S/E	FREQUENCY	TOTAL
public void setDropoffLocation(String dropoffLocation)	0	0	0
{	0	0	0
this.dropoffLocation = dropoffLocation;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getDriver()

CODE	S/E	FREQUENCY	TOTAL
public IDriver getDriver()	0	0	0
{	0	0	0
return driver;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setDriver()

CODE	S/E	FREQUENCY	TOTAL
public void setDriver(IDriver driver)	0	0	0
{	0	0	0
this.driver = driver;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

toString()

CODE	S/E	FREQUENCY	TOTAL
public String toString()	0	0	0
{	0	0	0
return "Ride Id: " + rideId + "\nPickup Location: " + pickupLocation + "\nDropOff Location: " + dropoffLocation + "\nPickup Time: " + pickupTime + "\nDropOff Time: " + dropoffTime + "\nDriver: " + driver;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

compareTo()

CODE	S/E	FREQUENCY	TOTAL
public int compareTo(IRide other)	0	0	0
{	0	0	0
return pickupLocation.compareTo(other.getPickupLocation());	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

PrivateRide

PrivateRide (constructor)

CODE	S/E	FREQUENCY	TOTAL
public PrivateRide(int rideId, String pickupLocation, IDateTime pickupTime, IDateTime dropoffTime, String dropoffLocation, IRider rider)	0	0	0
{	0	0	0
super(rideId, pickupLocation, pickupTime, dropoffTime, dropoffLocation);	1	1	1
this.rider = rider;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2**		
BIG O	**O(1)**		

hasRider()

CODE	S/E	FREQUENCY	TOTAL
public boolean hasRider(int riderId)	0	0	0
{	0	0	0
return rider.getId() == riderId;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

getRider()

CODE	S/E	FREQUENCY	TOTAL
public IRider getRider()	0	0	0
{	0	0	0
return rider;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

setRider()

CODE	S/E	FREQUENCY	TOTAL
public void setRider(IRider rider)	0	0	0
{	0	0	0
this.rider = rider;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

hasRider()

CODE	S/E	FREQUENCY	TOTAL
public boolean hasRider(String name)	0	0	0
{	0	0	0
return rider.getName().equals(name);	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

SharedRide

SharedRide (constructor)

CODE	S/E	FREQUENCY	TOTAL
public SharedRide(int rideId, String pickupLocation, IDateTime pickupTime, IDateTime dropoffTime, String dropoffLocation, RiderList participantList)	0	0	0
{	0	0	0
super(rideId, pickupLocation, pickupTime, dropoffTime, dropoffLocation);	1	1	1
this.participantList = participantList;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2**		
BIG O	**O(1)**		

SharedRide (constructor)

CODE	S/E	FREQUENCY	TOTAL
public SharedRide(int rideId, String pickupLocation, IDateTime pickupTime, IDateTime dropoffTime, String dropoffLocation)	0	0	0
{	0	0	0
super(rideId, pickupLocation, pickupTime, dropoffTime, dropoffLocation);	1	1	1
participantList = new RiderList();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 2**		
BIG O	**O(1)**		

hasRider()

CODE	S/E	FREQUENCY	TOTAL
public boolean hasRider(int riderId)	0	0	0
{	0	0	0
return participantList.findById(riderId) != null;	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

getParticipants()

CODE	S/E	FREQUENCY	TOTAL
public LinkedList<IRider> getParticipants()	0	0	0
{	0	0	0
return participantList.getAll();	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

addParticipant()

CODE	S/E	FREQUENCY	TOTAL
public boolean addParticipant(IRider rider)	0	0	0
{	0	0	0
return participantList.add(rider);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

removeParticipantById()

CODE	S/E	FREQUENCY	TOTAL
public boolean removeParticipantById(int riderId)	0	0	0
{	0	0	0
return participantList.removeById(riderId);	n	1	n
}	0	0	0
TOTAL OPERATIONS	**T(n) = n**		
BIG O	**O(n)**		

isEmpty()

CODE	S/E	FREQUENCY	TOTAL
public boolean isEmpty()	0	0	0
{	0	0	0
return participantList.size() <= 0;	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = 1**		
BIG O	**O(1)**		

hasRider()

CODE	S/E	FREQUENCY	TOTAL
public boolean hasRider(String name)	0	0	0
{	0	0	0
LinkedList<IRider> nameList = participantList.findByName(name);	n	1	n
return !nameList.empty();	1	1	1
}	0	0	0
TOTAL OPERATIONS	**T(n) = n + 1**		
BIG O	**O(n)**		